УЧЕБНИК ПО HTML

Теги и их атрибуты

Этот учебник — систематический обзор всех основных HTML-тегов и их атрибутов по стандарту HTML5. Каждая глава посвящена отдельной группе тегов: структуре документа, тексту, ссылкам, спискам, медиа, таблицам, формам, семантическим блокам и глобальным атрибутам.

Все примеры можно проверять в любом браузере: сохраните код в файл с расширением .html и откройте двойным щелчком. Никаких серверов и сборщиков не нужно — HTML работает прямо в браузере.

ОГЛАВЛЕНИЕ

- Глава 1. Структура HTML-документа
- Глава 2. Текстовые теги
- Глава 3. Ссылки
- Глава 4. Списки
- Глава 5. Медиа
- Глава 6. Таблицы
- Глава 7. Формы
- Глава 8. Семантические теги
- Глава 9. Глобальные атрибуты
- Глава 10. Практические задачи
- Глава 11. Итоги и рекомендации

ГЛАВА 1. СТРУКТУРА HTML-ДОКУМЕНТА

1.1. Что такое HTML

HTML (HyperText Markup Language) — язык разметки гипертекста. Это не язык программирования, а способ описать структуру документа: где заголовок, где абзац, где картинка, где ссылка. Браузер читает HTML и строит по нему страницу.

HTML состоит из тегов. Тег — это ключевое слово в угловых скобках, например (paragraph, абзац) или <img> (image, картинка). Большинство тегов парные: есть открывающий и закрывающий , между которыми помещается содержимое.□□ теги одиночные — у них нет закрывающей пары, потому что они ничего не содержат.

1.2. Атрибуты

Атрибуты — это дополнительные свойства тега, записанные в открывающем теге через пробел. Атрибут имеет вид имя="значение". Одному тегу можно задать несколько атрибутов в любом порядке:

```
<a href="https://example.com"
   target="_blank"
   rel="noopener">Открыть</a>
```

Здесь у тега <a> три атрибута: href (адрес), target (где открыть), rel (тип отношения). Значения обычно берутся в двойные кавычки, но одинарные тоже допустимы.

Булевы атрибуты могут записываться без значения — само их присутствие означает true. Например, disabled или required:

```
<input type="text" disabled required>
```

1.3. Минимальный каркас страницы

Любой HTML5-документ начинается с объявления <!DOCTYPE html> — оно говорит браузеру, что перед нами современный HTML, а не XHTML или HTML 4. Дальше идёт корневой тег <html>, внутри которого два раздела: <head> (метаданные) и <body> (видимое содержимое).

```
<!DOCTYPE html>
<html lang="ru">
  <head>
    <meta charset="utf-8">
    <meta name="viewport"
          content="width=device-width,
                   initial-scale=1">
    <title>Моя страница</title>
  </head>
  <body>
    <h1>Привет, мир!</h1>
    <p>Это мой первый HTML-документ.</p>
  </body>
</html>
```

1.4. <html lang>

Тег <html> — корневой, обёртывает весь документ. Атрибут lang указывает язык содержимого. Это важно для скринридеров (правильное произношение), поисковиков и браузерной проверки орфографии. Значения — коды по стандарту BCP 47:

- lang="ru" — русский
- lang="en" — английский
- lang="ar" — арабский (RTL по умолчанию)
- lang="he" — иврит (RTL по умолчанию)
- lang="zh-Hans" — китайский упрощённый
- lang="pt-BR" — португальский Бразилии

1.5. <head> и его содержимое

В разделе <head> лежат метаданные: всё, что не отображается на странице напрямую, но нужно браузеру и поисковикам.

- <meta charset="utf-8"> — кодировка документа
- <meta name="viewport" content="..."> — адаптивность
- <title> — заголовок вкладки браузера (обязателен)
- <meta name="description" content="..."> — описание для SEO
- <meta name="author" content="..."> — автор
- <link rel="stylesheet" href="style.css"> — подключение CSS
- <script src="app.js" defer></script> — подключение JS

Атрибут charset указывает кодировку. Современный стандарт — utf-8, поддерживает все языки мира. Если его не указать, браузер может выбрать кодировку по умолчанию и кириллица превратится в «кракозябры».

Тег viewport критически важен для мобильных устройств. Без него браузер телефона отображает страницу как на десктопе и уменьшает её, чтобы влезла в экран. С viewport, указанным как выше, страница адаптируется к ширине устройства.

1.6. <body>

Тег <body> содержит всё, что пользователь видит на странице: заголовки, абзацы, изображения, формы. Допускается только один <body> на документ.

ГЛАВА 2. ТЕКСТОВЫЕ ТЕГИ

2.1. Заголовки <h1> – <h6>

Шесть уровней заголовков, от <h1> (самый важный, обычно заголовок страницы) до <h6> (вложенный подраздел). Браузеры отображают их жирным шрифтом убывающего размера.

```
<h1>Название сайта</h1>
<h2>Название раздела</h2>
<h3>Подраздел</h3>
<h4>Пункт</h4>
<h5>Подпункт</h5>
<h6>Мельчайший уровень</h6>
```

Правила хорошего тона: на странице должен быть ровно один <h1> — это заголовок страницы. Дальше идёт иерархия: <h2> — разделы, <h3> — подразделы внутри <h2>, и так далее. Не пропускайте уровни: после <h2> сразу <h4> — ошибка. Поисковики и скринридеры используют иерархию для навигации.

2.2. Абзацы и переводы строк

Тег <p> (paragraph) выделяет абзац текста. Браузер автоматически добавляет отступы сверху и снизу. Тег
 (break) — одиночный, делает перенос строки внутри абзаца без нового абзаца.

```
<p>Первая строка абзаца.<br>
Вторая строка, но тот же абзац.</p>
<p>Это уже новый абзац.</p>
```

Не используйте
 для создания межстрочных интервалов — для этого есть CSS margin.
 оправдан только в стихах, адресах и других случаях, где перенос строки семантически значим.

2.3. <hr> — горизонтальная линия

Одиночный тег <hr> (horizontal rule) проводит горизонтальную линию и обозначает тематический разрыв — переход к новой теме в тексте. По умолчанию это серая линия во всю ширину.

```
<h2>Глава первая</h2>
<p>...</p>
<hr>
<h2>Глава вторая</h2>
```

2.4. <pre> и <code>

Тег <pre> (preformatted) сохраняет пробелы и переводы строк как есть, не схлопывая их. Используется для блоков кода, ASCII-графики, стихов с заданной разбивкой.

```
<pre>
function hello() {
    console.log('Привет');
}
</pre>
```

Тег <code> обозначает фрагмент кода внутри строки. Часто используется вместе с <pre> для блоков, но отдельно — для инлайновых упоминаний:

```
<p>Используйте функцию
  <code>parseInt()</code>
для преобразования строки в число.</p>
```

2.5. <blockquote> и <q>

Тег <blockquote> выделяет длинную цитату, обычно с отступом слева. Тег <q> — короткую цитату внутри строки, браузер автоматически добавляет кавычки.

```
<blockquote>
  <p>Всё, что мы есть, — это результат
     наших мыслей.</p>
  <footer>— Будда</footer>
</blockquote>

<p>Как сказал Нельсон: <q>Англия ждёт,
  что каждый выполнит свой долг.</q></p>
```

Атрибут cite у <blockquote> и <q> указывает URL источника цитаты. Не отображается браузером, но доступен машинам:

```
<blockquote cite="https://example.com/source">
  <p>...</p>
</blockquote>
```

2.6. Выделение и важность

Несколько тегов для смыслового выделения текста. Каждый несёт семантическую нагрузку, а не просто меняет внешний вид:

- — важное содержимое (жирный)
- <b> — привлечь внимание без доп. важности (жирный)
- — эмфатическое ударение (курсив)
- <i> — технический термин, фраза на другом языке (курсив)
- <mark> — выделенное как маркером
- <u> — комментарий/аннотация (подчёркивание)
- <s> — больше не актуально (зачёркивание)
- <small> — мелкий шрифт (юридический текст, дисклеймер)

```
<p><strong>Внимание:</strong> это
  <em>очень</em> важно!</p>
<p>Слово <i>пример</i> пришло
  из латыни.</p>
<p><mark>Этот фрагмент</mark>
  пользователь уже искал.</p>
```

Различие и <b> тонкое. говорит браузеру и скринридеру: это важная часть. <b> — просто жирный, без доп. смысла. Аналогично с /<i>: несёт эмфазу (голосовое ударение), <i> — стилистический курсив (термин, название корабля).

2.7. <sub>, <sup>, <abbr>, <time>

Теги для специальных текстовых ситуаций:

- <sub> — подстрочный текст (H_2O)
- <sup> — надстрочный текст ($E = mc^2$)
- <abbr title="..."> — аббревиатура с расшифровкой
- <time datetime="2026-10-26"> — дата/время для машин
- <kbd> — клавиша (Ctrl, Enter)
- <samp> — вывод программы
- <var> — переменная в коде

```
<p>Формула воды: H<sub>2</sub>O.</p>
<p>E = mc<sup>2</sup>.</p>
<p><abbr title="HyperText Markup Language">
  HTML</abbr> — язык разметки.</p>
<p>Дедлайн: <time datetime="2026-10-26">
  26 октября 2026</time>.</p>
```

Тег <time> особенно важен для семантики: поисковики и календари понимают, что здесь дата. Атрибут datetime принимает форматы ISO 8601: YYYY-MM-DD для даты, YYYY-MM-DDTHH:MM для момента времени.

2.8. Спецсимволы (entities)

Некоторые символы в HTML нужно экранировать, потому что они имеют специальный смысл: <, >, &. Используйте entities:

- < — меньше чем (<)
- > — больше чем (>)
- & — амперсанд (&)
-   — неразрывный пробел
- © — знак копирайта (©)
- — — длинное тире (—)
- … — многоточие (...)
- « » — кавычки-ёлочки («»)

```
<p>5 &amp;lt; 10 и 10 &amp;gt; 5.</p>
<p>Компания &amp;copy; 2026.</p>
<p>Длинное&amp;nbsp;слово не переносится.</p>
```

Неразрывный пробел   полезен, чтобы предлоги и союзы не оставались одни в конце строки: «в школе», «и т. д.» — эти связки переносятся целиком на новую строку.

ГЛАВА 3. ССЫЛКИ

3.1. Тег <a> и атрибут href

Тег <a> (anchor) создаёт гиперссылку. Атрибут href указывает URL, на который ведёт ссылка:

```
<a href="https://example.com">Перейти</a>
```

Содержимое тега — это то, что увидит пользователь: текст, картинка или другой HTML. По клику браузер переходит по указанному URL.

3.2. Абсолютные и относительные URL

Абсолютный URL содержит полный адрес, включая протокол и домен:

```
<a href="https://example.com/page">Абсолют</a>
<a href="//example.com/page">Без протокола</a>
<a href="/about">От корня сайта</a>
<a href="page.html">В той же папке</a>
<a href="../index.html">На уровень выше</a>
<a href="./about/team.html">В подпапку</a>
```

Форма //example.com без протокола — браузер подставит тот же протокол, что у текущей страницы (http или https). Удобно при разработке, но сегодня, когда почти везде HTTPS, явный https:// безопаснее.

3.3. Якоря и id

Ссылка может вести на часть той же страницы через якорь. Якорь — это значение id любого элемента, на который хотим сослаться:

```
<h2 id="contacts">Контакты</h2>
...
<!-- где-то ниже -->
<a href="#contacts">К контактам</a>

<!-- на другую страницу в нужное место -->
<a href="/about.html#team">Команда</a>
```

При клике браузер прокрутит страницу до элемента с указанным id. Это особенно удобно для длинных статей с оглавлением.

3.4. target — где открыть

Атрибут target указывает, где открыть ссылку:

- _self — в той же вкладке (по умолчанию)
- _blank — в новой вкладке
- _parent — в родительском фрейме
- _top — в полном окне, игнорируя фреймы

```
<a href="https://example.com"
   target="_blank"
   rel="noopener noreferrer">
  Открыть в новой вкладке
</a>
```

3.5. rel — отношение ссылки

Атрибут rel описывает тип связи между текущей страницей и целью ссылки. Самые частые значения:

- noopener — запретить новой вкладке доступ к window.opener
- noreferrer — не передавать referer на новый сайт
- nofollow — не передавать SEO-вес (для рекламных ссылок)
- sponsored — платная/рекламная ссылка
- ugc — пользовательский контент (комментарии)
- noopener noreferrer — комбо для target="_blank"
- icon — ссылка на иконку (в <link>)
- stylesheet — ссылка на CSS (в <link>)

Современное правило: всегда добавляйте rel="noopener noreferrer" к ссылкам с target="_blank". Иначе новая вкладка получит доступ к window.opener текущей и сможет её перенаправить (атака типа reverse tabnabbing). Современные браузеры по умолчанию включают noopener, но для надёжности указывайте явно.

3.6. Специальные href

Ссылка может не только вести на веб-страницу. Несколько специальных форматов href:

```
<!-- Email: откроет почтовый клиент -->
<a href="mailto:info@example.com">Написать</a>
<a href="mailto:info@example.com?subject=Вопрос">
  С темой</a>

<!-- Телефон: откроет звонилку -->
<a href="tel:+78001234567">Позвонить</a>

<!-- Скачать файл -->
<a href="/files/manual.pdf" download>
  Скачать</a>
<a href="/files/manual.pdf"
   download="manual-v2.pdf">С другим именем</a>
```

Атрибут download без значения просто предлагает сохранить файл. С значением — сохраняет под указанным именем. Работает только для файлов с того же домена (политика CORS).

ГЛАВА 4. СПИСКИ

4.1. Маркированный список

Тег <ul> (unordered list) создаёт список с маркерами. Каждый элемент оборачивается в <li> (list item):

```
<ul>
  <li>Хлеб</li>
  <li>Молоко</li>
  <li>Яйца</li>
</ul>
```

По умолчанию маркеры — точки. В современном HTML атрибут type для <ul> считается устаревшим: вид маркеров настраивается через CSS list-style-type. Семантически важен только сам факт «неупорядоченного» списка — порядок элементов не важен.

4.2. Нумерованный список

Тег <ol> (ordered list) создаёт нумерованный список — порядок элементов имеет значение:

```
<ol>
  <li>Налить воду в кастрюлю</li>
  <li>Довести до кипения</li>
  <li>Добавить соль</li>
  <li>Засыпать макароны</li>
</ol>
```

У <ol> три полезных атрибута:

- type="1" — арабские цифры (по умолчанию)
- type="A" — заглавные буквы (A, B, C...)
- type="a" — строчные буквы (a, b, c...)
- type="I" — римские заглавные (I, II, III...)
- type="i" — римские строчные (i, ii, iii...)
- start="5" — начать нумерацию с 5
- reversed — нумерация в обратном порядке

```
<ol type="A" start="3">
  <li>Третий пункт буквой C</li>
  <li>Четвёртый пункт буквой D</li>
</ol>

<ol reversed>
  <li>Считаем вниз</li>
  <li>От большего к меньшему</li>
</ol>
```

У <li> есть атрибут value — задаёт значение для конкретного пункта, и последующие пункты продолжат с него:

```
<ol>
  <li>Первый</li>
  <li value="5">Пятый</li>
  <li>Шестой (автоматом)</li>
</ol>
```

4.3. Список определений <dl>

Тег <dl> (description list) создаёт список пар термин-описание. Внутри — <dt> (description term) для термина и <dd> (description definition) для описания:

```
<dl>
  <dt>HTML</dt>
  <dd>Язык разметки гипертекста.</dd>

  <dt>CSS</dt>
  <dd>Каскадные таблицы стилей.</dd>

  <dt>JavaScript</dt>
  <dd>Язык программирования браузера.</dd>
</dl>
```

Браузеры отображают <dd> с отступом слева. Один <dt> может иметь несколько <dd> (несколько значений термина), и наоборот — несколько <dt> к одному <dd> (если у терминов общее определение).

4.4. Вложенные списки

Списки можно вкладывать друг в друга, помещая / внутрь <li>. Браузер автоматически меняет маркер внутреннего списка:

```
<ul>
  <li>Фрукты
    <ul>
      <li>Яблоки</li>
      <li>Груши</li>
    </ul>
  </li>
  <li>Овощи
    <ul>
      <li>Морковь</li>
      <li>Картофель</li>
    </ul>
  </li>
</ul>
```

Важная тонкость: внутренний <ul> должен находиться внутри <li>, а не сразу после него. Это правильный HTML, в отличие от часто встречающейся ошибки, когда вложенный список ставят между <li> на одном уровне.

ГЛАВА 5. МЕДИА

5.1. Тег <img>

Тег <img> вставляет изображение. Это одиночный тег — без закрывающей пары. Атрибут src указывает путь к файлу, alt — альтернативный текст для скринридеров и на случай, если изображение не загрузилось:

```
<img src="photo.jpg"
     alt="Закат над морем"
     width="800" height="600">
```

Атрибуты width и height — это внутренние размеры картинки в пикселях. Современные браузеры используют их, чтобы зарезервировать место под картинку до её загрузки: страница не «прыгает», когда картинка догружается. Всегда указывайте width и height.

Атрибут loading контролирует ленивую загрузку:

- loading="lazy" — загружать только когда картинка попадает в зону видимости
- loading="eager" — загружать сразу (по умолчанию)

Lazy loading экономит трафик и ускоряет первую отрисовку страницы, особенно на длинных страницах с множеством картинок.

5.2. Адаптивные изображения <picture>

Тег <picture> позволяет браузеру выбрать изображение в зависимости от условий: ширины экрана, плотности пикселей, поддержки форматов. Внутри несколько <source> и один <img> в качестве фолбэка:

```
<picture>
  <source srcset="photo.avif" type="image/avif">
  <source srcset="photo.webp" type="image/webp">
  <source media="(min-width: 800px)"
          srcset="photo-large.jpg">
  <img src="photo.jpg" alt="Закат"
       width="800" height="600">
</picture>
```

Браузер проходит <source> сверху вниз и берёт первый подходящий: если поддерживает AVIF — берёт AVIF, иначе WebP, иначе JPG. Атрибут media задаёт медиазапрос: разные картинки для разных экранов. Финальный <img> — обязателен, он же задаёт alt и размеры.

5.3. <figure> и <figcaption>

Тег <figure> выделяет самостоятельный блок контента — картинку, диаграмму, фрагмент кода — с подписью. Внутри может быть что угодно, а <figcaption> даёт подпись:

```
<figure>
  <img src="chart.png"
       alt="Продажи по месяцам">
  <figcaption>Рис. 1. Динамика продаж
    за первое полугодие 2026 года.</figcaption>
</figure>
```

Подпись может быть как до, так и после контента — оба варианта валидны. Главное преимущество <figure> — семантическая связь: поисковики и скринридеры понимают, что картинка и подпись — одно целое.

5.4. <video>

Тег <video> встраивает видеоплеер. Можно указать один src или несколько <source> для разных форматов:

```
<video src="movie.mp4" controls width="640">
  Ваш браузер не поддерживает video.
</video>

<!-- С несколькими источниками -->
<video controls width="640"
       poster="cover.jpg">
  <source src="movie.webm" type="video/webm">
  <source src="movie.mp4" type="video/mp4">
  Ваш браузер не поддерживает video.
</video>
```

Атрибуты <video>:

- controls — показать элементы управления
- autoplay — запускать автоматически (часто требует muted)
- muted — без звука
- loop — зациклить
- poster="url" — обложка до запуска
- preload="auto|metadata|none" — стратегия загрузки
- width / height — размеры плеера

5.5. <audio>

Тег <audio> встраивает аудиоплеер. Синтаксис такой же, как у <video>, только без poster и width/height:

```
<audio controls>
  <source src="song.ogg" type="audio/ogg">
  <source src="song.mp3" type="audio/mpeg">
  Ваш браузер не поддерживает audio.
</audio>
```

5.6. <iframe>

Тег <iframe> (inline frame) встраивает другую страницу внутрь текущей. Часто используется для YouTube, карт, виджетов:

```
<iframe src="https://www.youtube.com/embed/abc123"
        width="560" height="315"
        title="Видео"
        allowfullscreen></iframe>
```

Важные атрибуты безопасности:

- sandbox — ограничения для встроенного документа

- allow — какие API разрешены (camera, geolocation, ...)
- allowfullscreen — разрешить переход в полный экран
- loading="lazy" — ленивая загрузка
- title — описание для скринридеров (важно!)

Атрибут sandbox без значений включает все ограничения. Чтобы разрешить что-то конкретное, перечислите через пробел:

```
<iframe src="widget.html"
        sandbox="allow-scripts allow-same-origin">
</iframe>
```

ГЛАВА 6. ТАБЛИЦЫ

6.1. Базовая структура

Таблица собирается из нескольких вложенных тегов. <table> обёртывает всё, <tr> (table row) — строка, <td> (table data) — ячейка, <th> (table header) — заголовок колонки или строки:

```
<table>
  <tr>
    <th>Имя</th>
    <th>Возраст</th>
  </tr>
  <tr>
    <td>Анна</td>
    <td>28</td>
  </tr>
  <tr>
    <td>Иван</td>
    <td>34</td>
  </tr>
</table>
```

6.2. <thead>, <tbody>, <tfoot>

Для семантики и стилизации таблицу делят на три части:

- <thead> — шапка с заголовками колонок
- <tbody> — основное тело таблицы
- <tfoot> — подвал (например, итоги)

```
<table>
  <thead>
    <tr><th>Товар</th><th>Цена</th></tr>
  </thead>
  <tbody>
    <tr><td>Хлеб</td><td>50</td></tr>
    <tr><td>Молоко</td><td>80</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Итого</td><td>130</td></tr>
  </tfoot>
</table>
```

Если thead/tbody/tfoot не указать, браузер сам оборачивает все <tr> в неявный <tbody>. Но явное указание улучшает читаемость и позволяет стилизовать секции через CSS :nth-child.

6.3. <caption>

Тег <caption> даёт таблице заголовок. Должен быть первым ребёнком <table>:

```
<table>
  <caption>Цены на товары, руб.</caption>
  <thead>...</thead>
  <tbody>...</tbody>
</table>
```

6.4. colspan и rowspan

Атрибут colspan объединяет несколько ячеек по горизонтали, rowspan — по вертикали:

```
<table>
  <tr>
    <th colspan="2">Данные пользователя</th>
  </tr>
  <tr>
    <td>Имя</td>
    <td>Анна</td>
  </tr>
  <tr>
    <td rowspan="2">Контакты</td>
    <td>anna@example.com</td>
  </tr>
  <tr>
    <td>+7 800 123-45-67</td>
  </tr>
</table>
```

В первой строке одна ячейка с colspan=2 занимает ширину двух колонок. Во третьей строке ячейка с rowspan=2 растягивается на две строки. Это позволяет строить сложные таблицы-шахматки, расписание и т.д.

6.5. Атрибут scope у <th>

Атрибут scope у <th> указывает, к чему относится заголовок — к строке или колонке. Это помогает скринридерам корректно озвучивать таблицу:

- scope="col" — заголовок колонки
- scope="row" — заголовок строки
- scope="colgroup" — для группы колонок
- scope="rowgroup" — для группы строк

```
<table>
  <tr>
    <th scope="col">Имя</th>
    <th scope="col">Возраст</th>
  </tr>
  <tr>
    <th scope="row">Анна</th>
    <td>28</td>
  </tr>
</table>
```

Не используйте атрибут border для <table> — он устарел в HTML5. Рамки настраиваются через CSS border и border-collapse.

ГЛАВА 7. ФОРМЫ

7.1. <form>

Тег <form> обёртывает все элементы формы. Атрибуты указывают, куда и как отправлять данные:

```
<form action="/submit" method="post">
  <!-- поля формы -->
  <button type="submit">Отправить</button>
</form>
```

Атрибуты <form>:

- action="URL" — куда отправить данные

- method="get"|"post" — метод HTTP
- enctype — тип кодирования (для файлов)
- target — где показать результат (_blank, _self)
- novalidate — отключить браузерную валидацию

Различие GET и POST: GET добавляет данные в URL (?name=Anna&age=28), подходит для фильтров и поиска. POST отправляет данные в теле запроса, невидимы в URL, подходит для изменения данных и загрузки файлов. Для загрузки файлов enctype обязательно multipart/form-data.

7.2. <input> — типы

Тег <input> — самый универсальный элемент формы. Его поведение зависит от атрибута type:

- type="text" — однострочный текст (по умолчанию)
- type="email" — email с проверкой формата
- type="password" — пароль (символы скрыты)
- type="number" — число со стрелками
- type="tel" — телефон (на мобильном открывает цифровую клаву)
- type="url" — URL с проверкой
- type="search" — поисковое поле
- type="date" — дата (календарь)
- type="time" — время
- type="datetime-local" — дата и время
- type="month", type="week" — месяц/неделя
- type="color" — выбор цвета
- type="range" — ползунок
- type="checkbox" — галочка
- type="radio" — переключатель (один из группы)
- type="file" — загрузка файла
- type="hidden" — скрытое поле (не видно, отправляется)
- type="submit" — кнопка отправки
- type="reset" — кнопка сброса
- type="button" — обычная кнопка (нужен JS)
- type="image" — кнопка-картинка

```
<input type="email" name="email"
       placeholder="you@example.com" required>

<input type="number" name="age" min="1" max="120"
       value="18">

<input type="checkbox" name="agree" checked>

<input type="file" name="avatar" accept="image/*">
```

7.3. Атрибуты <input>

Самые ходовые атрибуты полей ввода:

- name — имя поля (станет ключом в отправленных данных)
- value — текущее значение
- placeholder — подсказка в пустом поле
- required — обязательное для заполнения
- disabled — отключено (серое, не отправляется)
- readonly — только для чтения (отправляется)
- autofocus — курсор при загрузке страницы
- autocomplete="on|off" — автозаполнение браузера
- min, max — границы для number/date/range
- step — шаг для number/range
- pattern="regex" — регулярка для валидации
- maxlength, minlength — длина текста
- multiple — несколько значений (для email/file)
- accept — фильтр типов файлов (для type=file)

```
<input type="text" name="username"
       pattern="[A-Za-z]{3,20}"
       title="Латинские буквы, 3-20 символов"
       required>

<input type="file" name="photos" multiple
       accept="image/jpeg,image/png">
```

7.4. <label>

Тег <label> связывает подпись с полем. Это важно для доступности и UX — клик по label переводит фокус на поле. Связь через атрибут for, ссылающийся на id поля:

```
<label for="email">Email</label>
<input type="email" id="email" name="email">

<!-- Альтернатива: поле внутри label -->
<label>
  Email
  <input type="email" name="email">
</label>
```

Второй вариант (поле внутри label) тоже валидный и не требует id/for. Удобен для чекбоксов и радиокнопок, где клик по подписи должен переключать состояние:

```
<label>
  <input type="checkbox" name="agree">
  Я согласен с условиями
</label>
```

7.5. <textarea>

Многострочное текстовое поле. В отличие от <input>, имеет открывающий и закрывающий теги, между которыми — содержимое по умолчанию:

```
<textarea name="comment" rows="5" cols="40"
          maxlength="500" placeholder="Ваш комментарий...">
Текст по умолчанию.
</textarea>
```

Атрибуты rows и cols задают видимый размер в строках и колонках. Реальный размер лучше контролировать через CSS width/height. resize: vertical разрешает пользователю тянуть только по высоте — частый паттерн.

7.6. <select>, <option>, <optgroup>

Выпадающий список. <select> обёртывает <option> — пункты:

```
<label for="country">Страна</label>
<select name="country" id="country">
  <option value="">— выберите —</option>
  <option value="ru">Россия</option>
  <option value="by" selected>Беларусь</option>
  <option value="kz">Казахстан</option>
</select>
```

Атрибут selected выбирает пункт по умолчанию. Атрибут value — что отправится на сервер; если не указан, отправится текст option. <optgroup> группирует пункты:

```
<select name="city">
  <optgroup label="Россия">
    <option value="msk">Москва</option>
    <option value="spb">Санкт-Петербург</option>
  </optgroup>
  <optgroup label="Беларусь">
    <option value="minsk">Минск</option>
  </optgroup>
</select>

<!-- Множественный выбор -->
<select name="tags" multiple size="5">
  <option value="html">HTML</option>
  <option value="css">CSS</option>
</select>
```

С multiple пользователь выбирает несколько пунктов через Ctrl/Cmd+клик. size задаёт число видимых пунктов.

7.7. <button>

Тег <button> — кнопка. Универсальнее <input type=submit>, потому что может содержать HTML (иконки, форматирование):

```
<button type="submit">Отправить</button>
<button type="reset">Сбросить</button>
<button type="button">Не отправлять</button>

<!-- С иконкой -->
<button type="submit">
  <img src="icon.svg" alt=""> Сохранить
</button>
```

type="submit" (по умолчанию в форме) — отправляет форму. type="reset" — сбрасывает. type="button" — ничего не делает без JS; используется для кастомных действий.

7.8. <fieldset> и <legend>

Тег <fieldset> группирует поля формы, рисуя рамку. <legend> — подпись группы:

```
<fieldset>
  <legend>Личные данные</legend>
  <label>Имя: <input type="text" name="name"></label>
  <label>Возраст: <input type="number" name="age"></label>
</fieldset>

<fieldset>
  <legend>Контакты</legend>
  <label>Email: <input type="email" name="email"></label>
</fieldset>
```

7.9. <datalist>

Тег <datalist> даёт подсказки для текстового поля — список предлагаемых значений, из которых пользователь может выбрать или ввести своё:

```
<label for="browser">Любимый браузер</label>
<input list="browsers" id="browser" name="browser">
<datalist id="browsers">
  <option value="Chrome">
  <option value="Firefox">
  <option value="Safari">
  <option value="Edge">
</datalist>
```

Связь через атрибут list у <input> и id у <datalist>. Похоже на <select>, но пользователь может ввести произвольный текст, не только выбрать из списка.

ГЛАВА 8. СЕМАНТИЧЕСКИЕ ТЕГИ

8.1. Зачем нужна семантика

В HTML4 сайты верстали на с классами вроде class="header". Браузер, скринридер, поисковик — все видели одинаковую мешанину div-ов. HTML5 добавил семантические теги, которые сразу говорят, что именно здесь находится. Это важно для SEO, доступности и удобства поддержки кода.

8.2. Структурные семантические теги

- <header> — шапка страницы или раздела
- <footer> — подвал, копирайт, контакты
- <nav> — навигация по сайту (меню)
- <main> — основное содержимое (одно на страницу)

- <section> — тематический раздел документа
- <article> — самостоятельная статья (пост, новость)
- <aside> — боковая панель, связанный контент

```
<body>
  <header>
    <h1>Мой блог</h1>
    <nav>
      <a href="/">Главная</a>
      <a href="/about">О блоге</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>Заголовок поста</h2>
      <p>Текст...</p>
    </article>
  </main>

  <aside>
    <h3>Свежие посты</h3>
    <ul>...</ul>
  </aside>

  <footer>
    © 2026 Мой блог
  </footer>
</body>
```

Правило: <main> только один на страницу. <header> и <footer> могут быть у страницы и у отдельных <article> или <section>. <nav> использовать для основной навигации, не для всех ссылок подряд.

8.3. <article> vs <section>

Граница между этими тегами не всегда очевидна. Правило: <article> — это самостоятельная единица контента, которую можно вырвать из страницы и опубликовать отдельно (пост в блоге, новость, карточка товара). <section> — тематическая группа внутри документа, чаще всего с собственным заголовком.

Если сомневаетесь, спросите: можно ли этот блок перенести на другую страницу как самостоятельную статью? Да — <article>. Нет, это часть большего — <section>.

8.4. <time>

Тег <time> выделяет дату или время. Атрибут datetime задаёт машиночитаемое значение, а содержимое — то, что пользователь видит:

```
<article>
  <h2>Встреча клуба</h2>
  <p>Состоится <time datetime="2026-11-15">
    15 ноября 2026 года</time>
    в <time datetime="2026-11-15T19:00">
    19:00</time>.</p>
</article>
```

Форматы datetime по ISO 8601:

- Год-месяц-день: 2026-11-15
- С временем: 2026-11-15T19:00
- С таймзоной: 2026-11-15T19:00+03:00
- Только время: 19:00
- Только год-месяц: 2026-11

8.5. <address>

Тег <address> выделяет контактную информацию — автора статьи или владельца сайта. Браузеры отображают курсивом:

```
<footer>
  <address>
    Автор: Иван Петров<br>
    Email: <a href="mailto:ivan@example.com">
      ivan@example.com</a><br>
    Телефон: <a href="tel:+78001234567">
      +7 800 123-45-67</a>
  </address>
</footer>
```

8.6. <details> и <summary>

Тег <details> создаёт раскрывающийся блок без JavaScript. <summary> — видимая подпись, по клику раскрывает остальное:

```
<details>
  <summary>Часто задаваемые вопросы</summary>
  <p>Здесь текст с ответами на вопросы.</p>
  <p>Можно любой HTML внутри.</p>
</details>

<!-- Раскрыт по умолчанию -->
<details open>
  <summary>Дополнительная информация</summary>
  <p>...</p>
</details>
```

Отличное решение для FAQ, спойлеров, расширенных настроек. Атрибут open раскрывает блок при загрузке.

8.7. <dialog>

Тег <dialog> создаёт модальное окно без JS-библиотек. Атрибут open показывает его сразу:

```
<dialog open>
  <p>Вы уверены?</p>
  <button>Да</button>
  <button>Отмена</button>
</dialog>
```

Для полноценного управления (открытие по клику, закрытие по Esc) нужен метод .showModal() в JavaScript. Сам тег даёт встроенную обработку фокуса и бекдропа.

ГЛАВА 9. ГЛОБАЛЬНЫЕ АТРИБУТЫ

9.1. Что такое глобальные атрибуты

Глобальные атрибуты — те, что можно применить к любому HTML-тегу. В отличие от специализированных (href у <a>, src у <img>), они работают везде.

9.2. id и class

id — уникальный идентификатор элемента. На странице не должно быть двух элементов с одинаковым id. Используется для якорей (#id в ссылках), JavaScript (getElementById) и точечных стилей:

```
<h2 id="contacts">Контакты</h2>

<!-- CSS -->
<style>#contacts { color: blue; }</style>

<!-- JS -->
<script>
  document.getElementById('contacts')
    .scrollIntoView();
</script>
```

class — стилевой класс. Можно повторять, можно давать несколько элементам, можно несколько классов одному элементу через пробел:

```
<p class="lead text-muted">Текст</p>
<p class="lead">Другой текст</p>

<!-- CSS -->
<style>
  .lead { font-size: 1.25rem; }
  .text-muted { color: gray; }
</style>
```

9.3. style

Атрибут style задаёт инлайновые CSS-свойства. Имеет высший приоритет (после !important), поэтому перекрывает стили из <style> и внешних файлов:

```
<p style="color: red; font-weight: bold;">
  Красный жирный текст
</p>
```

Инлайновые стили — плохая практика, кроме случаев, когда стили генерируются динамически (значение из базы, позиционирование drag-and-drop и т.д.). В обычных случаях используйте class.

9.4. title

Атрибут title добавляет всплывающую подсказку при наведении. Полезен для дополнительных пояснений, но не основное место для важной информации (скринридеры читают title неодинаково):

```
<a href="/help" title="Откроется в новом окне"
   target="_blank">Помощь</a>

<abbr title="World Wide Web">WWW</abbr>
```

9.5. lang — язык фрагмента

Атрибут lang можно применять не только к <html>, но и к любому элементу — для фрагментов текста на другом языке. Это помогает скринридерам правильно произносить текст и проверке орфографии выбирать словарь:

```
<p>Английский термин <span lang="en">
  hyperlink</span> означает ссылку.</p>

<!-- Французская фраза в русском тексте -->
<p>Как говорят французы: <q lang="fr">
  C'est la vie</q>.</p>
```

9.6. dir — направление текста

Атрибут dir указывает направление текста. Большинство языков пишут слева направо (LTR), но арабский, иврит, персидский, урду — справа налево (RTL). Значения:

- dir="ltr" — слева направо (по умолчанию)
- dir="rtl" — справа налево
- dir="auto" — браузер сам определит по первому символу

Атрибут можно применять к любому элементу. Глобально его задают на <html>, а локально — на конкретные фрагменты RTL-текста внутри LTR-страницы:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
  <head><meta charset="utf-8"></head>
  <body>
    <h1>مرحبا بالعالم</h1>
    <p>هذا نص باللغة العربية.</p>
  </body>
</html>
```

Когда dir="rtl" задан на <html>, браузер не только пишет текст справа налево, но и зеркалит многие элементы интерфейса: flexbox и grid меняют направление, прокрутка появляется слева, иконки со стрелками «назад/вперёд» меняются местами. CSS-свойства margin-left/right в RTL автоматически становятся логическими margin-inline-start/end через inline-start, если использовать современные логические свойства.

9.7. dir в смешанном тексте

В документе с преимущественно LTR-текстом могут быть RTL-вставки. Используйте dir на конкретном элементе, а для одной RTL-строки — dir="auto":

```
<html lang="ru" dir="ltr">
  <body>
    <p>Арабская фраза:
      <span lang="ar" dir="rtl">
        السلام عليكم
      </span>
      – привет.
    </p>

    <!-- Пользовательский ввод,
         направление неизвестно -->
    <p><bdi dir="auto">123أدمح</bdi>
       добавил комментарий.</p>
  </body>
</html>
```

Тег <bdi> (bidirectional isolation) изолирует фрагмент, чтобы его направление не влияло на окружающий текст. Полезен для пользовательских имён и других данных с неизвестным направлением. В связке с dir="auto" браузер сам определяет направление по первому сильному символу.

9.8. data-* атрибуты

Префикс data- позволяет создавать произвольные атрибуты для хранения данных. Они доступны в JavaScript через dataset:

```
<button data-action="delete"
        data-id="42"
        data-confirm="Удалить?">Удалить</button>

<script>
  const btn = document.querySelector('button');
  console.log(btn.dataset.action);  // "delete"
  console.log(btn.dataset.id);      // "42"
  console.log(btn.dataset.confirm); // "Удалить?"
</script>
```

Имена data- должны быть в нижнем регистре. data-user-id в JS превратится в dataset.userId (camelCase). Значения — строки; для чисел и объектов делайте JSON.parse вручную.

9.9. tabindex

Атрибут tabindex управляет порядком перемещения по Tab-клавише:

- tabindex="0" — элемент доступен по Tab в естественном порядке
- tabindex="-1" — можно фокусировать только через JS focus()
- tabindex="1" — явный порядок (избегайте чисел > 0)

Положительные tabindex считаются плохой практикой — они ломают естественный порядок фокуса. Используйте tabindex="0" для доступности кастомных элементов (например,) и tabindex="-1" для программного фокуса.

9.10. hidden, contenteditable, spellcheck

- hidden — скрыть элемент (как display:none)
- contenteditable="true|false" — делает элемент редактируемым
- spellcheck="true|false" — включить/выключить проверку орфографии
- translate="yes|no" — переводить ли текст в автопереводчиках
- draggable="true|false|auto" — можно ли перетаскивать

```
<div contenteditable spellcheck="true">
  Этот текст можно редактировать прямо в браузере.
</div>

<p hidden>Этот абзац не виден.</p>

<p translate="no">Brand Name Inc.</p>
```

ГЛАВА 10. ПРАКТИЧЕСКИЕ ЗАДАЧИ

10.1. Задача 1: структура блога

Создайте семантическую структуру страницы блога: шапка с навигацией, основной контент с одной статьёй и сайдбаром, подвал с контактами. Решение:

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Мой блог</title>
</head>
<body>
  <header>
    <h1>Мой блог</h1>
    <nav>
      <a href="/">Главная</a>
      <a href="/about">О блоге</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>Заголовок поста</h2>
      <time datetime="2026-10-26">
        26 октября 2026</time>
      <p>Текст поста...</p>
    </article>
  </main>

  <aside>
    <h3>Обо мне</h3>
    <p>Короткое описание автора.</p>
  </aside>

  <footer>
    <address>ivan@example.com</address>
    <p>© 2026 Мой блог</p>
  </footer>
</body>
</html>
```

10.2. Задача 2: форма регистрации

Сделайте форму регистрации: имя, email, пароль, согласие с условиями. С валидацией. Решение:

```
<form action="/register" method="post">
  <fieldset>
    <legend>Регистрация</legend>

    <p>
      <label for="name">Имя</label>
      <input type="text" id="name"
             name="name" required
             minlength="2" maxlength="50">
    </p>

    <p>
      <label for="email">Email</label>
      <input type="email" id="email"
             name="email" required>
    </p>

    <p>
      <label for="pass">Пароль</label>
      <input type="password" id="pass"
             name="password" required
             minlength="8"
             pattern="[A-Za-z0-9]{8,}">
    </p>

    <p>
      <label>
        <input type="checkbox"
               name="agree" required>
        Согласен с условиями
      </label>
    </p>

    <button type="submit">Зарегистрироваться</button>
  </fieldset>
</form>
```

10.3. Задача 3: таблица расписания

Создайте таблицу расписания на неделю. Используйте thead/tbody, scope, colspan. Решение:

```
<table>
  <caption>Расписание на неделю</caption>
  <thead>
    <tr>
      <th scope="col">Время</th>
      <th scope="col">Пн</th>
      <th scope="col">Вт</th>
      <th scope="col">Ср</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">09:00</th>
      <td>Математика</td>
      <td rowspan="2">Физкультура</td>
      <td>Русский</td>
    </tr>
    <tr>
      <th scope="row">10:00</th>
      <td>История</td>
      <td>Литература</td>
    </tr>
    <tr>
      <th scope="row">11:00</th>
      <td colspan="3">Большая лекция</td>
    </tr>
  </tbody>
</table>
```

10.4. Задача 4: карточка товара

Создайте карточку товара с картинкой, названием, ценой, рейтингом и кнопкой «в корзину». Решение:

```
<article>
  <h2>Ноутбук XYZ</h2>
  <picture>
    <source srcset="laptop.webp"
            type="image/webp">
    <img src="laptop.jpg"
         alt="Ноутбук XYZ, вид спереди"
         width="400" height="300"
         loading="lazy">
  </picture>
  <p>Цена: <strong>50 000 ₽</strong></p>
  <p>Рейтинг:
    <span aria-label="4 из 5">★★★★☆</span>
  </p>
  <form action="/cart" method="post">
    <input type="hidden" name="id" value="42">
    <label>Кол-во:
      <input type="number" name="qty"
             value="1" min="1" max="99">
    </label>
    <button type="submit">В корзину</button>
  </form>
</article>
```

10.5. Задача 5: адаптивное изображение

Сделайте изображение, которое загружает AVIF, если поддерживается, иначе WebP, иначе JPG. С разными размерами под экран. Решение:

```
<picture>
  <source
    media="(min-width: 1200px)"
    srcset="photo-xl.avif 1920w
            photo-xl.webp 1920w
            photo-xl.jpg  1920w"
    sizes="100vw">
  <source
    media="(min-width: 600px)"
    srcset="photo-m.avif 800w
            photo-m.webp 800w
            photo-m.jpg  800w"
    sizes="100vw">
  <source type="image/avif"
          srcset="photo-s.avif 400w">
  <source type="image/webp"
          srcset="photo-s.webp 400w">
  <img src="photo-s.jpg"
       alt="Описание"
       width="400" height="300"
       loading="lazy"
       decoding="async">
</picture>
```

10.6. Задача 6: RTL-страница на арабском

Создайте минимальную страницу на арабском языке с правильным направлением. Внутри — RTL-цитата на иврите. Решение:

```
<!DOCTYPE html>
<html lang="ar" dir="rtl">
<head>
  <meta charset="utf-8">
  <meta name="viewport"
        content="width=device-width,
                 initial-scale=1">
  <title>مرحبا</title>
</head>
<body>
  <header>
    <h1>مرحبا بالعالم</h1>
    <nav>
      <a href="/">الرئيسية</a>
      <a href="/about">حول</a>
    </nav>
  </header>

  <main>
    <p>هذا نص باللغة العربية.</p>

    <!-- Цитата на иврите внутри -->
    <blockquote lang="he" dir="rtl">
      <p>□□□□ □□□□</p>
    </blockquote>
  </main>
</body>
</html>
```

Ключевые моменты: dir="rtl" на <html> переводит весь документ в правостороннее направление. Браузер автоматически зеркалит отступы, выравнивание по умолчанию, порядок пунктов в навигации. lang="ar" помогает скринридерам правильно произносить текст.

10.7. Задача 7: семантическая шапка

Создайте шапку сайта с логотипом, поиском, переключателем темы и меню. Решение:

```
<header>
  <a href="/" aria-label="Главная">
    <img src="logo.svg" alt="Логотип"
         width="120" height="40">
  </a>

  <nav aria-label="Основная навигация">
    <ul>
      <li><a href="/products">Товары</a></li>
      <li><a href="/blog">Блог</a></li>
      <li><a href="/contacts">Контакты</a></li>
    </ul>
  </nav>

  <form role="search" action="/search">
    <label for="q" class="visually-hidden">
      Поиск
    </label>
    <input type="search" id="q"
           name="q" placeholder="Найти...">
    <button type="submit">Найти</button>
  </form>

  <button type="button"
          data-action="toggle-theme"
          aria-label="Переключить тему">
    
  </button>
</header>
```

Атрибут aria-label на <nav> важен, если на странице несколько навигаций — он помогает скринридеру различить их. Класс visually-hidden обычно определяется в CSS как визуально скрытый, но доступный элемент (clip: rect(0,0,0,0); position: absolute).

ГЛАВА 11. ИТОГИ И РЕКОМЕНДАЦИИ

11.1. Итоги по главам

Глава 1. HTML — язык разметки, не программирования. Теги бывают парные и одиночные. Атрибуты — пары имя="значение". Минимальный каркас: DOCTYPE, html, head, body.

Глава 2. <h1>–<h6> — заголовки, <p> — абзац,
 — перенос, <hr> — тематический разрыв. / — семантическое выделение, <code>/<pre> — код. Спецсимволы экранируются через &имя;.

Глава 3. <a href> — ссылки. Абсолютные и относительные URL, якоря по id, target=_blank с rel=noopener noreferrer, mailto:/tel:/download для спец-ссылок.

Глава 4. /<ol>/ — списки, type/start/reversed/value для нумерации, <dl>/<dt>/<dd> — список определений. Вложенные списки вкладываются в <li>.

Глава 5. <img> — картинки с alt и width/height, <picture>+<source> для адаптивных, <figure>+<figcaption> для подписи, <video>/<audio> с controls, <iframe> с sandbox.

Глава 6. <table> с <thead>/<tbody>/<tfoot>, <caption> для заголовка, colspan/rowspan для объединения ячеек, scope у <th> для доступности.

Глава 7. <form action method> обёртывает поля. <input> имеет десятки type, <label for> связывает подпись, <select>+<option> — выпадающий список, <textarea> — многострочный текст, <fieldset>+<legend> группирует.

Глава 8. Семантические теги <header>/<footer>/<nav>/<main>/<section>/<article>/<aside> говорят браузеру и поисковикам, что где находится. <time>, <address>, <details>, <dialog> для специфичных случаев.

Глава 9. Глобальные атрибуты: id, class, style, title, lang, dir, data-*, tabindex, hidden, contenteditable, spellcheck. Особенно важен dir для RTL-языков — арабского, иврита, персидского.

Глава 10. Семь практических задач показали, как комбинировать теги из предыдущих глав: структура блога, форма регистрации, таблица расписания, карточка товара, адаптивное изображение, RTL-страница, семантическая шапка.

11.2. Что читать дальше

После освоения этого учебника логично двигаться в трёх направлениях. Во-первых, CSS — язык описания внешнего вида HTML-документа. Без него страницы выглядят как серый текст 90-х.

Во-вторых, спецификация MDN Web Docs (developer.mozilla.org) — лучший справочник по каждому тегу и атрибуту. Там указаны все возможные значения, поддержка браузерами, примеры и интерактивные песочницы.

В-третьих, валидатор W3C (validator.w3.org) — проверяет HTML на соответствие стандарту. Запускайте свой код через него после каждой большой страницы — это находит опечатки, незакрытые теги, устаревшие атрибуты.

Главный совет: не пытайтесь заучить все теги и атрибуты. HTML — это практика. Сделайте свой сайт-визитку, страницу для события, документацию к домашнему проекту. С каждой новой строкой кода приходит интуитивное понимание, когда нужен <article>, а когда <section>, когда <figure>, а когда просто <img>. Опыт складывается из десятков сделанных и переделанных страниц, а не из прочитанных справочников.
